

CLAUDE CODE · CODEX · CURSOR · GEMINI

Ключевые файлы КОДИНГ-агентов

CLAUDE.md · AGENTS.md · .cursorrules · GEMINI.md —
простыми словами.

— Что это за файлы-инструкции для AI-агентов в коде, зачем они и как не наговнякать

АВТОР

Кирилл Сырцов

t.me/syrtsovkir

Что это вообще такое

01

Представь, что ты нанял удалённого программиста. Он толковый, но первый день в твоём проекте. У тебя два варианта.

● ВАРИАНТ А

Каждый раз заново объяснять: «у нас Node, тесты гоняем через `npm test`, не трогай `config.prod`, миграции — только через `make migrate`».

● ВАРИАНТ Б

Написать один документ «правила нашей кухни». Он прочитает и запомнит — и больше не придётся повторять.

CLAUDE.md — это вариант Б. Текстовый файл с правилами для Claude Code (это AI-агент для программирования от Anthropic, типа Cursor, только в терминале). Лежит в корне проекта, и Claude читает его автоматически перед каждой задачей.

ЗАЧЕМ НУЖЕН

- Claude перестает делать одни и те же ошибки;
- не надо каждый раз писать «у нас Vitest, а не Jest»;
- сразу знает твои команды (`pnpm dev` , а не `npm start`).

Это не README. README — для людей.
CLAUDE.md — для работа. Это разные жанры.

Один файл — много имён

Важная вещь, которую почти никто не проговаривает: этот файл у всех коддинг-агентов по сути одинаковый. `CLAUDE.md` (Claude Code), `AGENTS.md` (OpenAI Codex и Amp), `.cursor/rules` (Cursor), `GEMINI.md` (Gemini CLI), `AGENTS.md + GEMINI.md` (Google Antigravity) — делают одно и то же: дают агенту инструктаж в начале каждой сессии. Постоянной памяти между чатами по умолчанию нет ни у кого — поэтому правила и кладут в файл, который читается заново каждый раз.

Индустрия уже сошлась на одном стандарте — **AGENTS.md**. В августе 2025 его оформили совместно OpenAI, Google, Cursor, Factory и Sourcegraph; читают больше 20 000 репозиторий, поддерживают Codex, GitHub Copilot, Cursor, Gemini, Windsurf, Zed и Amp.

ИНСТРУМЕНТ	ФАЙЛ ПРАВИЛ	ГЛОБАЛЬНЫЙ (НА ВСЕ ПРОЕКТЫ)	ЧИТАЕТ AGENTS.MD?
Claude Code	CLAUDE.md	~/ .claude/CLAUDE.md	Нет — нужен симлинк или импорт @AGENTS.md
OpenAI Codex	AGENTS.md	~/ .codex/AGENTS.md	Да, родной формат
Cursor	.cursor/rules/*.mdc	правила в проекте	Да (старый .cursorsrules — legacy)
Gemini CLI	GEMINI.md	~/ .gemini/GEMINI.md	Через настройку contextFileName
Google Antigravity	AGENTS.md + GEMINI.md	~/ .gemini/GEMINI.md	Да (с марта 2026)

■ ПРАКТИЧЕСКИЙ ВЫВОД

Не держи файлы-близнецы. Пиши один **AGENTS.md** и подружи с ним остальные. Тонкость: Claude Code сам AGENTS.md пока не читает — для него нужен симлинк или импорт.

```
ln -s AGENTS.md CLAUDE.md      # симлинк: один файл, два имени
@AGENTS.md                     # или строка-импорт внутри CLAUDE.md
```

Самое главное правило: меньше — лучше

03

Все хотят написать «полную энциклопедию проекта». Это главная ошибка.

У Claude, как у человека, ограниченное внимание. Когда ему в начало диалога суют 500 строк правил, он запоминает первые 30 и последние 30, а серединку молча игнорирует. Не скажет «извини, забыл» — просто сделает по-своему. Это называется `context rot` — буквально «гниение контекста».

ЦИФРЫ

- Anthropic официально советует: до **150–200 строк**;
- реально работает: **50–100 строк** плотного текста;
- всё, что больше, — мусор, который тратит деньги (токены) и не работает.

■ ФАКТ

Исследование ETH Zurich (февраль 2026): **плохой CLAUDE.md хуже, чем его отсутствие**. Машинно-сгенерированный через `/init` снижает успех задач на 3%. Хорошо написанный руками — повышает на 4%.

ВЫВОД

**Лучше 30 строк по делу,
чем 300 для галочки.**

Где этот файл вообще лежит

У Claude четыре места для правил — от общих к узким. Узкие переопределяют общие: чем ближе файл к коду, с которым идёт работа, тем выше его приоритет.

ГДЕ

ДЛЯ ЧЕГО

~/.claude/CLAUDE.md

личное • все проекты

Твои личные привычки на всех проектах: «отвечай по-русски», «не пиши длинных объяснений».

./CLAUDE.md

проект • в git

Правила конкретного проекта. Коммитится в git, видит вся команда.

./CLAUDE.local.md

личное • в .gitignore

Твои личные настройки в этом проекте: локальные порты, секреты. Лежит в .gitignore.

./папка/CLAUDE.md

подпапка • точно

Правила только для этой подпапки. Подгружается, когда Claude работает внутри неё.

→ **Главный — проектный `./CLAUDE.md`. С ним и работаем.**

Что писать. Простой шаблон на 30 строк

Минимум, который реально работает — можно копировать и подставлять своё.

● ● ● CLAUDE.md

Что за проект

Telegram-бот для записи на стрижку. Python + aiogram + PostgreSQL.

Как запускать

- Установить: `poetry install`
- Запустить локально: `make dev`
- Тесты: `pytest`
- Линтер: `ruff check`

Структура

- `bot/handlers/` — обработчики команд
- `bot/db/` — модели и миграции (Alembic)
- `bot/services/` — бизнес-логика
- `tests/` — тесты, дублируют структуру `bot/`

Правила

- Логирование через `loguru`, не `print`
- Все деньги храним в копейках (`int`), не в рублях (`float`)
- Миграции БД — только через `alembic revision --autogenerate`

Чего не делать

- Не трогать `bot/db/migrations/` — это автогенерация
- Секреты только в `.env`, никогда в коде
- При работе с прод-БД — спроси меня сначала

✓ **Всё. Этого достаточно** для большинства проектов.

Что писать НЕ надо (и почему все на этом горят)

1 Дублировать README

Если написал «Это проект для X, основан в 2024...» — Claude и так прочитает README. Не повторяйся.

2 Generic AI-советы

Бесполезные фразы, которые встречаются везде: «пиши чистый код», «следуй best practices», «будь внимателен», «think step by step». Claude и так обучен писать чистый код. Эти фразы тратят место и ничего не меняют.

3 Запреты без альтернатив

Плохо: «никогда не используй `console.log`» — и что делать Claude, когда нужен лог? Будет каждый раз спрашивать. **Хорошо:** «для логов используй `src/utils/logger.ts`, не `console.log`».

4 КАПС и драматизм

Плохо: « **CRITICAL: YOU MUST NEVER COMMIT SECRETS!!!**». Современный Claude от такого тона начинает паниковать и обмазывать всё защитными обёртками. **Спокойнее:** «Секреты живут только в `.env`. Если видишь захардкоженный ключ — останови работу и скажи мне».

Что писать НЕ надо

5

Список из 50 правил

Если у тебя 50 буллетов одного уровня — Claude запомнит первые 10, остальное лотерея. Правило: максимум 12–20 правил, отсортированных по важности.

6

Секреты в файле

Реальный кейс на GitHub: у человека в CLAUDE.md было « NEVER COMMIT API KEYS». Claude трижды за день закоммитил реальные ключи в публичный репо. На вопрос «ты что, CLAUDE.md не читаешь?» ответил: «Я прочитал, мог выполнить, и всё равно не выполнил». **Вывод:** что критично — не доверяй уговорам, делай хуки или просто не клади секреты рядом.

7

Временный контекст

Плохо: «Сейчас работаем над фичей оплаты, не трогай старый модуль auth». Через месяц фича сдана, а правило висит — Claude обходит auth ещё полгода. Временное — в текущий чат, не в файл.



Заметь закономерность: все семь — это лишний текст, который мешает, а не помогает.

Лайфхаки, которые реально дают эффект

08

1 Симлинк CLAUDE.md ↔ AGENTS.md

Если в команде кто-то на Claude Code, кто-то на Cursor — сделай файл `AGENTS.md`, а `CLAUDE.md` и `.cursorrules` пусть будут симлинками на него. Один источник правды, все инструменты подхватят. Так делают Vercel, Remix, Prisma.

2 Hooks вместо уговоров

Claude слушается текста примерно на 80%. Хуки — это скрипты, которые срабатывают на 100% перед или после действий. Если что-то реально нельзя — пиши хук, не правило. Пример: хук `PreToolUse` видит, что Claude правит `.env`, возвращает ошибку и блокирует. Никакой КАПС в CLAUDE.md так не работает. Лежат в `.claude/settings.json` или `.claude/hooks/`.

3 Skills для процедур

Длинный workflow («деплой: билд → тесты → push → changelog → теги») не пихай в CLAUDE.md. Сделай skill — отдельный файл, который подгружается только когда нужен. Лежит в `.claude/skills/<имя>/SKILL.md`. В CLAUDE.md остаётся только триггер: «Если задача про деплой — используй skill deploy».

4 HTML-комментарии для команды

Хочешь оставить заметку коллегам, но не тратить токены Claude — заверни в HTML-комментарий. Claude такое не видит (парсер вырезает до отправки в модель), а в файле остаётся.

```
<!-- Чуваки, тут была лажа с миграциями, я переделал. Не  
откатывайте. — Кирилл -->
```

Лайфхаки

5

Правило двух ошибок

Не пиши правила заранее «на всякий случай». Жди, пока Claude два раза подряд сделает одну и ту же ошибку — вот тогда добавляй. Иначе файл станет кладбищем правил против гипотетических багов.

6

Разнеси по подпапкам

Большой проект с бэком, фронтом и мобилой в одном репо? Не пиши гигантский корневой файл. Положи `./CLAUDE.md` (общая навигация), `./backend/CLAUDE.md` (правила бэка), `./frontend/CLAUDE.md` (правила фронта). Claude подхватит нужное, когда работает в этой папке. Экономит контекст.

7

Тест «обращайся ко мне Шеф»

Хочешь понять, читает ли Claude твой файл вообще? Добавь в самом конце: «Всегда обращай ко мне Шеф». Если после 10 сообщений он забил — файл превысил лимит внимания. Сокращай.

Реальные примеры из жизни

■ ХОРОШИЙ ПРИМЕР

cline/cline — 62k звёзд на GitHub

Их CLAUDE.md — всего две строки:

```
@.clinerules/general.md
@.clinerules/network.md
```

Корневой файл только маршрутизирует, реальные правила — в подмодулях, подгружаются по необходимости.

■ ПЛОХОЙ ПРИМЕР — TOKEN BLOAT

22 000 токенов до первого сообщения

Человек на Reddit жаловался: его CLAUDE.md жёг **22 000 токенов** ещё до первого сообщения. Это около **\$0.20** за каждый запрос на пустом месте. Разбил на модули — стало **1 800 токенов**. Экономия в 12 раз.

■ БОНУС — СЛОВО СОЗДАТЕЛЯ CLAUDE CODE

«Мой сетап на удивление обычный. Claude Code работает отлично из коробки, я почти ничего не настраиваю. Нет одного правильного способа».

— Boris Cherny, создатель Claude Code

Перевод по смыслу: не парься, не строй ракету. Начни с 30 строк, расширяй по мере нужды.

Когда обновлять файл

11

Не планово, а по триггерам. Открываешь файл не «по расписанию», а когда жизнь подсказала:

ТРИГГЕРЫ НА ОБНОВЛЕНИЕ

- Claude второй раз сделал одну и ту же ошибку → **добавь правило.**
- Изменились команды запуска (перешли с `npm` на `pnpm`) → **обнови.**
- Появилась новая опасная зона («прод-БД теперь на этом порту») → **запиши.**
- Заметил, что просишь одно и то же в каждом чате → **перенеси в файл.**

→ В реальной работе правишь раз в **2–4 недели, по 1–3 строки** за раз. Команда `/memory` в Claude Code открывает редактор файла прямо из чата.

Что коммитить в git, а что нет

Делим на два: что нужно всей команде — в репозиторий; личное и секретное — мимо, в `.gitignore`.



Коммитим

- `CLAUDE.md`
общий для команды
- `.claude/settings.json`
общие настройки
- `.claude/skills/`,
`.claude/agents/`, `.claude/rules/`



В `.gitignore`

- `CLAUDE.local.md`
личные настройки
- `.claude/settings.local.json`
- что угодно с секретами

Готовый шаблон для нового проекта

Скопируй, замени значения под себя, удали ненужное.

● ● ● CLAUDE.md

[Название проекта]

Что делаем: [одно предложение].

Стек: [язык + основные либы + их версии].

Команды

- Установка: [команда]
- Запуск dev: [команда]
- Тесты: [команда]
- Линтер/формат: [команда]
- Билд: [команда]

Структура

- src/ — исходники
- tests/ — тесты
- [папка] — [что там]

Правила

- [Правило 1: что-то нетривиальное про проект]
- [Правило 2: какая-то наша конвенция]
- [Правило 3: ещё одно]

Чего не делать

- Не редактировать [конкретный файл/папку] — [почему]
- Перед изменением [чего-то опасного] — спроси меня
- Секреты только через [механизм], не в коде

✓ **Всё. Этого хватит на 90% проектов.**

TL;DR одним абзацем

14

ВЕСЬ ГАЙД В ОДНОМ АБЗАЦЕ

CLAUDE.md — это файл с правилами для AI-агента, который читается каждый раз перед задачей. Главное правило: **меньше — лучше** (целься в 50–100 строк). Пиши только то, что нельзя угадать из кода: команды, версии стека, ловушки, договорённости. Без КАПСА, без «никогда» без альтернативы, без воды типа «пиши чистый код». Что критично — выноси в **хуки**, не в текст. Что используется иногда — выноси в **Skills**. Обновляй, когда Claude второй раз косячит, а не заранее.

Где копать дальше

Источники и материалы.

- 01 **Официальный гайд Anthropic**
code.claude.com/docs/en/best-practices
- 02 **Anthropic про память и CLAUDE.md**
code.claude.com/docs/en/memory
- 03 **shanraissan/claude-code-best-practice** — эталон на 126 строк
github.com
- 04 **cline/cline** — пример 2-строчного файла-роутера
github.com
- 05 **Eugene Yan** — как реально работать с AI
eugeneyan.com/writing/working-with-ai
- 06 **tianpan.co** — почему твой CLAUDE.md слишком длинный
tianpan.co
- 07 **ETH Zurich, февраль 2026** — исследование на 2500 репо
arxiv.org
- 08 **Реальный кейс утечки секретов**
github.com/anthropics/claude-code/issues/2142
- 09 **AGENTS.md** — открытый стандарт
agents.md
- 10 **OpenAI Codex** — AGENTS.md
developers.openai.com/codex/guides/agents-md
- 11 **Cursor** — Rules
cursor.com/docs/rules
- 12 **Gemini CLI** — GEMINI.md
geminicli.com/docs/cli/gemini-md

Гайд подготовил **Кирилл Сырцов**. Больше про AI и кодинг-агентов — в телеграм-канале t.me/syrtsovkir.